



UNIVERSIDADE FEDERAL DO PIAUÍ - UFPI
CENTRO DE CIÊNCIAS DA NATUREZA - CCN
DEPARTAMENTO DE COMPUTAÇÃO - DC
DOCENTE: VINICIUS PONTE MACHADO
DISCIPLINA: INTELIGÊNCIA ARTIFICIAL

TRABALHO PRÁTICO 01

Giovanna Maria Rodrigues Moita
Camilly Larissa Costa Silva
João Marcelo Barros Cardoso Ventura
Antonio Inácio da Silva Neto

Teresina, Piauí
2026

1. Introdução.....	3
2. Métodos de busca.....	3
2.1 Força Bruta.....	3
2.2 Busca Gulosa.....	3
2.3 A*.....	3
3. Metodologia.....	4
3.1 Algoritmos de busca.....	4
3.2 Interface.....	4
4. Resultados.....	5
4.1 Jogo do 8 (3x3).....	5
4.2 Jogo do 15 (4x4).....	6
5. Conclusão.....	7

1. Introdução

A busca em espaços de estados e o uso de heurísticas são fundamentais na resolução de problemas complexos em Inteligência Artificial. Este relatório apresenta o desenvolvimento de uma solução para o Jogo do Oito em um tabuleiro 3x3, com variação para 4x4.

O objetivo é analisar o comportamento dos algoritmos de Busca em Largura (BFS), Busca em Profundidade (DFS), Busca Gulosa e A* , avaliando métricas de desempenho como tempo de execução, custo do caminho, consumo de memória e a profundidade máxima atingida em cada estratégia. Ao final, discute-se a eficácia de cada método em termos de otimalidade e completude.

2. Métodos de busca

2.1 Força Bruta

Os métodos de força bruta, também chamados de buscas cegas, não utilizam conhecimento prévio sobre o problema além da definição dos estados possíveis e do estado objetivo. Dessa forma, o algoritmo explora o espaço de estados sem qualquer estimativa de proximidade da solução.

A Busca em Largura (Breadth-First Search – BFS) realiza a exploração nível por nível, visitando inicialmente todos os estados mais próximos da raiz antes de avançar para níveis mais profundos. Essa abordagem garante completude e também otimalidade quando o custo de cada movimento é uniforme, como ocorre no Jogo do Oito, já que todos os deslocamentos possuem o mesmo custo. Entretanto, seu principal problema está no elevado consumo de memória, pois muitos nós permanecem armazenados simultaneamente na fronteira.

Já a Busca em Profundidade (Depth-First Search – DFS) explora um ramo do espaço de estados até o máximo possível antes de retroceder. Seu uso de memória é significativamente menor quando comparado à BFS, pois mantém menos nós armazenados na fronteira. No entanto, a DFS não garante encontrar a solução ótima e, dependendo da profundidade da busca, pode inclusive não encontrar solução em espaços muito extensos ou com ciclos, caso não haja mecanismos adequados de controle.

Neste trabalho, ambas foram utilizadas como referência de busca cega para comparação com os métodos heurísticos.

2.2 Busca Gulosa

A Busca Gulosa (Greedy Search) utiliza uma função heurística para estimar o quão próximo um estado está da solução, escolhendo sempre o nó aparentemente mais promissor no momento. Diferentemente da BFS e da DFS, essa abordagem não considera o custo já percorrido, mas apenas a estimativa de proximidade até o objetivo.

A heurística adotada foi a Distância de Manhattan, que calcula a soma das distâncias horizontais e verticais de cada peça até sua posição correta no estado final, representando de forma consistente o número mínimo de deslocamentos necessários para posicionar cada peça corretamente.

Embora a Busca Gulosa geralmente encontre soluções mais rapidamente e com menor número de nós visitados do que os outros métodos usados, ela não garante otimalidade, pois pode escolher caminhos aparentemente vantajosos no curto prazo que resultam em soluções maiores no final.

2.3 A*

O algoritmo A* combina as vantagens da busca de custo uniforme com o uso de heurísticas, sendo amplamente reconhecido como uma das estratégias mais eficientes para problemas de busca em espaço de estados.

Sua decisão é baseada na função:

$$f(n)=g(n)+h(n)$$

em que $g(n)$ representa o custo já percorrido desde o estado inicial até o nó atual, e $h(n)$ representa a estimativa heurística, Distância de Manhattan, até o objetivo.

A função considera simultaneamente o custo acumulado e a estimativa futura, assim o A* consegue equilibrar exploração e eficiência, geralmente encontrando a solução ótima com menor custo computacional que a BFS. Quando a heurística utilizada é admissível, isto é, não superestima o custo real da solução, o algoritmo preserva sua propriedade de otimalidade.

Por esse motivo, o A* costuma apresentar melhor desempenho geral quando comparado às demais abordagens utilizadas neste trabalho.

3. Metodologia

O desenvolvimento da solução foi dividido em dois módulos principais: a implementação dos algoritmos de busca e a interface gráfica.

3.1 Algoritmos de busca

O arquivo `jogo8.py` concentra toda a lógica de resolução do problema com os métodos principais de busca e funções auxiliares fundamentais para o funcionamento da solução, responsáveis pela representação dos nós, geração de novos estados e cálculo da heurística utilizada.

Inicialmente, foi criada a classe `Node`, responsável por representar cada estado do tabuleiro durante a execução, onde cada nó armazena o estado atual, uma referência para o nó pai e a profundidade em que se encontra na árvore de busca. Essa estrutura permite reconstruir o caminho completo da solução por meio do método `get_path()`, que retorna a sequência de estados desde o estado inicial até o objetivo.

Em seguida, a função `generate_children()` foi desenvolvida para gerar todos os estados sucessores possíveis a partir de um determinado tabuleiro. Isso ocorre com a identificação da posição do espaço vazio (representado por 0) e cálculo dos movimentos válidos com base no tamanho do tabuleiro, o que permite compatibilidade tanto com o jogo 3x3 (Jogo do Oito) quanto com a variação 4x4 (Jogo do Quinze).

Outra função auxiliar é a `calculate_manhattan()`, responsável pelo cálculo da heurística utilizada na Busca Gulosa e no algoritmo A*. Como ambas as estratégias utilizam a mesma função heurística, sua implementação foi centralizada como uma função auxiliar independente.

Por fim, as estratégias de busca implementadas foram:

- `breadth_first_search()` para Busca em Largura;
- `depth_first_search()` para Busca em Profundidade;
- `greedy_search()` para Busca Gulosa;
- `a_star_search()` para o algoritmo A*.

Cada uma dessas abordagens foi projetada para retornar não apenas a solução encontrada, mas também métricas de desempenho exigidas no trabalho, como número de nós visitados, tamanho máximo da fronteira e profundidade máxima atingida. Essas informações foram utilizadas posteriormente para análise comparativa por meio de um sistema de atualização de progresso durante a execução, permitindo acompanhar em tempo real o comportamento da busca na interface gráfica.

3.2 Interface

O arquivo `interface.py` foi desenvolvido com a biblioteca Tkinter e tem como principal objetivo facilitar a inserção do estado inicial do problema, além de apresentar visualmente os resultados obtidos por cada algoritmo.

A interface permite ao usuário selecionar o tamanho do tabuleiro entre 3x3 e 4x4, inserir manualmente os valores das peças ou utilizar estados iniciais previamente configurados por meio de

presets. Também é possível escolher quais algoritmos serão executados individualmente ou em conjunto.

Após o início da execução, a interface exibe o progresso da busca em tempo real, mostrando informações como número de nós visitados, tamanho máximo da fronteira e tempo decorrido, o que melhora a análise prática do comportamento de cada algoritmo. Além disso, ao final da execução, os resultados são organizados em uma tabela comparativa contendo:

- Número de movimentos da solução;
- Nós visitados;
- Tamanho máximo da fronteira;
- Profundidade da solução;
- Profundidade máxima atingida;
- Tempo de execução;
- Indicação de completude;
- Indicação de otimalidade.

Além disso, foi implementada uma aba de visualização passo a passo da solução encontrada, permitindo acompanhar cada movimento realizado até o estado objetivo. Dessa forma, a metodologia adotada buscou não apenas resolver o problema proposto, mas também fornecer uma ferramenta de análise prática e comparativa entre diferentes estratégias de busca em Inteligência Artificial.

4. Resultados

Foram feitos diversos testes para observar de forma prática as diferenças e semelhanças de comportamento entre os algoritmos de busca aplicados ao Jogo do Oito (3x3) e ao Jogo do Quinze (4x4).

4.1 Jogo do 8 (3x3)

4.1.1 Teste 1

Entrada: [1, 2, 3, 4, 0, 5, 7, 8, 6]

1	2	3
4	0	5
7	8	6

Saída:

Algoritmo	Movimentos	Nós visitados	Máx. fronteira	Prof. solução	Prof. máx.	Tempo (s)	Ótimo	Completo
BFS (Largura)	2	13	8	2	3	0.0001	Sim	Sim
DFS (Profundidade)	2	3	5	2	2	0.0000	Sim	Sim
Gulosa	2	3	5	2	2	0.0000	Sim	Sim
A*	2	3	5	2	2	0.0000	Sim	Sim

Neste teste, o estado inicial encontra-se bastante próximo da solução final, exigindo poucas movimentações para alcançar o objetivo, por isso, todos os algoritmos conseguiram encontrar a solução rapidamente e apresentando métricas bem próximas.

4.1.2 Teste 2

Entrada: [1, 0, 3, 4, 2, 6, 7, 5, 8]

1	0	3
4	2	6
7	5	8

Saída:

Algoritmo	Movimentos	Nós visitados	Máx. fronteira	Prof. solução	Prof. máx.	Tempo (s)	Ótimo	Completo
BFS (Largura)	3	11	10	3	4	0.0001	Sim	Sim
DFS (Profundidade)	13893	14295	10940	13893	13893	0.0567	Não	Sim
Gulosa	3	4	6	3	3	0.0001	Sim	Sim
A*	3	4	6	3	3	0.0001	Sim	Sim

Neste caso, o problema exige uma sequência um pouco maior de movimentos, permitindo observar melhor as diferenças entre os algoritmos. A BFS garantiu a solução ótima, porém com maior número de nós visitados, a DFS mostrou comportamento menos previsível, gerando soluções maiores dependendo da profundidade explorada e a Busca Gulosa e o A* tiveram bom desempenho, reduzindo a quantidade de estados explorados e preservando a otimalidade.

4.1.3 Teste 3

Entrada: [0, 1, 3, 5, 2, 6, 4, 7, 8]

0	1	3
5	2	6
4	7	8

Saída:

Algoritmo	Movimentos	Nós visitados	Máx. fronteira	Prof. solução	Prof. máx.	Tempo (s)	Ótimo	Completo
BFS (Largura)	6	78	57	6	7	0.0003	Sim	Sim
DFS (Profundidade)	47620	137749	42913	47620	66126	0.5757	Não	Sim
Gulosa	6	7	7	6	6	0.0001	Sim	Sim
A*	6	7	7	6	6	0.0001	Sim	Sim

Este teste foi o mais complexo entre os casos do tabuleiro 3x3, exigindo uma busca mais profunda no espaço de estados. Nesse cenário, ficou mais evidente a limitação da BFS em relação ao consumo de memória, já que o número de nós visitados cresceu significativamente comparado aos testes anteriores e a DFS apresentou ainda mais instabilidade na qualidade da solução. Já os algoritmos heurísticos novamente se destacaram ao conseguirem encontrar a solução ideal com menor esforço computacional, demonstrando melhor escalabilidade para problemas mais difíceis.

4.2 Jogo do 15 (4x4)

4.2.1 Teste 1

Entrada: [1,2,3,4,5,6,7,8,9,10,11,12,13,14,0,15]

1	2	3	4
5	6	7	8
9	10	11	12
13	14	0	15

Saída:

Algoritmo	Movimentos	Nós visitados	Máx. fronteira	Prof. solução	Prof. máx.	Tempo (s)	Ótimo	Completo
BFS (Largura)	1	4	6	1	2	0.0000	Sim	Sim
DFS (Profundidade)	1	2	3	1	1	0.0000	Sim	Sim
Gulosa	1	2	3	1	1	0.0001	Sim	Sim
A*	1	2	3	1	1	0.0000	Sim	Sim

Neste primeiro teste do tabuleiro 4x4, o estado inicial está muito próximo da solução, sendo necessário apenas um movimento para atingir o objetivo. Todos os algoritmos apresentaram excelente desempenho, com tempo de resposta praticamente imediato. Esse teste serviu principalmente para validar o funcionamento correto da generalização da implementação para o Jogo do Quinze, mostrando que a lógica utilizada para o 3x3 também se adapta corretamente ao 4x4.

4.2.2 Teste 2

Entrada: [1,2,3,4,5,6,7,8,9,10,11,12,13,0,14,15]

1	2	3	4
5	6	7	8
9	10	11	12
13	0	14	15

Saída:

Algoritmo	Movimentos	Nós visitados	Máx. fronteira	Prof. solução	Prof. máx.	Tempo (s)	Ótimo	Completo
BFS (Largura)	2	10	14	2	3	0.0001	Sim	Sim
DFS (Profundidade)	2	3	4	2	2	0.0000	Sim	Sim
Gulosa	2	3	4	2	2	0.0001	Sim	Sim
A*	2	3	4	2	2	0.0001	Sim	Sim

Neste segundo teste, o aumento da complexidade, mesmo que pouco, tornou mais evidente a diferença entre buscas cegas e buscas heurísticas. A BFS passou a exigir mais memória devido ao crescimento exponencial do espaço de estados, enquanto a DFS, a Busca Gulosa e o algoritmo A* apresentaram grande velocidade e conseguiram encontrar soluções ótimas com menor custo computacional.

5. Conclusão

Este trabalho permitiu analisar, de forma prática, o desempenho de diferentes algoritmos de busca aplicados ao Jogo do Oito e ao Jogo do Quinze. A comparação entre BFS, DFS, Busca Gulosa e A* mostrou que cada estratégia apresenta vantagens e limitações dependendo do objetivo e da complexidade do problema.

A BFS garantiu soluções ótimas, mas com alto consumo de memória, enquanto a DFS utilizou menos memória, porém sem garantir o melhor caminho. A Busca Gulosa apresentou maior rapidez em vários testes, mas nem sempre encontrou a solução ideal. Já o algoritmo A* demonstrou o melhor equilíbrio entre eficiência e otimalidade, principalmente nos casos mais complexos, como o tabuleiro 4x4. Além disso, a heurística de Distância de Manhattan mostrou-se adequada para orientar os métodos heurísticos, reduzindo o número de estados explorados e melhorando o desempenho geral.

Dessa forma, conclui-se que, para problemas maiores e mais complexos, o uso de buscas heurísticas torna-se a alternativa mais eficiente, evidenciando sua importância na resolução de problemas em Inteligência Artificial.